

Concurrent Task Dispatcher

Final Project Design Report

Damian Villarreal

May 2026

Overview

This project implements a concurrent task dispatcher in Rust. CPU-bound and IO-bound tasks arrive over time, sit in dual ready queues, and are dispatched to a fixed pool of eight worker threads. Two scheduling policies (a FIFO baseline and a weighted policy) are run against an identical workload to compare their behavior.

Components

The program runs five long-lived threads:

- **Generator** (1): Builds the full task list from a fixed RNG seed before any other thread starts, ensuring runs are reproducible. Sleeps until each task's arrival time and pushes it into the appropriate ready queue. Sets a ``closed`` flag and notifies the dispatcher after the final task.
- **Dispatcher** (1): Sleeps on a condition variable until either queue has work or the input is closed. Selects which queue to draw from according to the policy, then sends the task to a specific worker over that worker's ``mpsc`` channel.
- **Workers** (8): Each owns one ``mpsc`` receiver. Pulls a task, simulates execution, sends a ``Completion`` record to the metrics thread, and loops. Exits when its channel closes.
- **Metrics** (1): Single receiver of ``Completion`` records over an ``mpsc`` channel with cloned senders. Returns the full list once all workers exit and the channel closes.
- **Monitor** (1): Independent of the dispatch path. Every 10 ms it reads ``/proc/stat`` for system CPU usage and an ``AtomicUsize`` counter that workers increment on task start and decrement on completion. Samples are written to a CSV; summary statistics are returned at the end.

Shared state and synchronization

Two ready queues: ``VecDeque<Task>`` for CPU and another for IO. They sit behind a single ``Mutex`` paired with a ``Condvar``. A single mutex was chosen over two because the policy must examine both queues atomically (e.g., comparing the head wait times across queues).

Two mutexes would require holding both for every dispatch decision, which just was too complicated for me to implement, and one mutex worked just fine.

The ``active_workers`` counter is an ``Arc<AtomicUsize>``. A mutex is unnecessary because a 10 ms-stale read is acceptable for monitoring, and locking would only add contention.

The metrics ``Vec<Completion>`` is owned solely by the metrics thread; all other threads communicate through the channel. The channel itself provides synchronization, so no additional lock is needed.

Channels versus shared state

Channels are used for ownership transfer: tasks moving from generator to queues (via a method that internally locks), tasks from dispatcher to a chosen worker (one ``mpsc`` per worker), and completion records from workers to the metrics thread (one ``mpsc`` with cloned senders).

Shared state is used where multiple threads must examine the same data on different schedules: the queues themselves, because the dispatcher must compare across them to make a policy decision; and the active-worker counter, because the monitor reads it independently of when workers update it.

Scheduling policy

The dispatcher's ``choose_queue()`` function applies three rules in priority order:

1. **Aging override:** If either queue's head has waited longer than ``aging_threshold_ms`` (200 ms in the optimized run), serve that queue regardless of weights. This is the starvation guard.
2. **Single-queue fallback:** If only one queue has work, serve it.
3. **Weighted alternation:** A counter cycles through ``cpu_weight + io_weight`` ticks. The first ``cpu_weight`` ticks dispatch CPU; the remainder dispatch IO.

The FIFO experiment uses ``cpu_weight = io_weight = 1``, producing a strict CPU/IO alternation in which the aging branch never fires. The optimized experiment uses ``cpu_weight = 1, io_weight = 2``, producing a CPU, IO, IO, CPU, IO, IO ... cycle.

Why the optimized policy outperforms FIFO

The workload is 70% IO and 30% CPU, but CPU tasks are roughly five times longer (300-500 ms versus 50-100 ms). The CPU class therefore has more total work-time despite having fewer tasks. Strict alternation dispatches CPU and IO at the same rate, which mismatches the workload: the IO queue drains quickly, leaving some workers idle while others remain blocked on CPU work. Aggregate utilization stays low.

The 1:2 weighting in the optimized policy approximately matches the work-time ratio of the two classes, keeping all workers busy with a more balanced mix. The aging threshold guarantees that the long IO bias cannot starve the CPU queue once IO tasks drain in the tail of the run.

Tradeoffs

The optimized policy reduces total runtime, increases average CPU usage, and narrows the gap between worker finish times. The cost is that the policy is tuned to this workload's CPU/IO mix; under a different mix (for example, 50/50 with equal-length tasks), the 1:2 weighting would over-favor IO. The aging mechanism partially compensates. It would fire as the CPU queue built up, but the weights themselves remain workload-specific. An adaptive policy that adjusts the ratio at runtime would probably be what's next for a project like this.

A bug encountered during development

The first version of the program hung at shutdown: workers and dispatcher exited cleanly, but the process did not terminate. The cause was that the metrics thread was blocked indefinitely in ``recv()``. An ``mpsc::Receiver`` only returns ``Err`` when *all* senders are dropped, and the original ``metrics_tx`` was still alive in the function that spawned the threads. It had been cloned for each worker but never explicitly dropped.

The fix was a single line: ``drop(metrics_tx)`` immediately after the worker-spawn loop. The broader takeaway is that with ``mpsc``, channel closure is defined by the disappearance of every sender, not the exit of any specific thread. Each channel in the program now has an explicit answer to where its last sender is dropped: for the worker task channels, at the end of the dispatcher's run loop; for the metrics channel, when the last worker thread exits.

Residual starvation risk

The aging check inspects only the head of each queue. A task three positions deep behind two long CPU tasks could wait the full duration of those tasks before reaching the head and

triggering aging. Per-task wait tracking would catch this case earlier, at the cost of a peek at every task's enqueue time on every dispatch decision.

A second risk is policy mistuning: if the workload's CPU/IO mix differs substantially from this experiment's, the static weights are wrong and tasks of the disfavored class could pile up before aging fires.

Experiment results

If you run my project, you'll notice that I tried to make my results as close to the examples provided on Brightspace. Of course, with a focus on runtime, as that is the main metric we are aiming for. Both experiments use the same fixed seed, the same 1,000-task workload (706 IO tasks at 50-100 ms, 294 CPU tasks at 300-500 ms, all arriving in a burst at $t = 0$), and the same eight workers. The only variable is the dispatch weighting.

Metric	FIFO	Optimized	Change
Total runtime	34.0s	22.2s	-35%
Average CPU usage	~62%	~96%	+34 pts!! 😊
Avg active workers	~5 of 8	7.7 of 8	More even
Avg wait (CPU)	0.1s	1.7s	longer
Fairness gap	3.3s	0.8s	smaller

The "fairness gap" row captures a subtler effect. Under FIFO, IO tasks finish almost immediately because the dispatcher serves them at the same rate they arrive, while CPU tasks accumulate; the gap between average CPU and IO wait times is therefore large. Under the optimized policy, IO tasks wait longer in absolute terms, but CPU waits drop, and the two classes converge. The improvement is in the distribution of wait time across classes, not in the absolute IO experience.